

Stage 4: Operating Expenses, rule by rule

MekHQ implements this section more completely than the rulebook describes it — and gets the hard parts right. The three worked examples on pp. 25–28 reproduce exactly against the code. What diverges is narrow, specific, and mostly **fixable in an afternoon**; what is missing is mostly missing on purpose.

11 FAITHFUL	5 EXCEEDS RAW	9 DIVERGES	5 NOT IMPLEMENTED
----------------	------------------	---------------	----------------------

Validation against the book's own worked examples

Before judging anything, I ran Arnold's, Hannah's and Jason's completed worksheets through the code's formulas by hand. Every figure below is printed in the book; the right-hand column is what MekHQ computes.

UNIT	QUANTITY	BOOK	MEKHQ FORMULA	
TDR-5S Thunderbolt (65 t)	Spare parts	0.065 t / 650 CB	$65 \times 0.001 \times 10000$	match
BNC-3M Banshee (95 t)	Spare parts	0.095 t / 950 CB	$95 \times 0.001 \times 10000$	match
Bulldog (60 t)	Spare parts	0.060 t / 480 CB	$60 \times 0.001 \times 8000$	match
CSR-V12 Corsair (50 t)	Spare parts	0.050 t / 750 CB	$50 \times 0.001 \times 15000$	match
Seeker DropShip (6,700 t)	Spare parts	6.7 t / 100,500 CB	$6700 \times 0.001 \times 15000$	match
Merchant JumpShip (120,000 t)	Spare parts	12 t / 180,000 CB	$120000 \times 0.0001 \times 15000$	match
Foot laser platoon	Spare parts	0.006 t / 60 CB	$3 \times 0.002 \times 10000$	match
Bulldog (23 t engine)	Fuel	9.2 t petrochem	$23 \times 0.1 \times 4$	match
Scorpion (6 t engine)	Fuel	2.4 t petrochem	$6 \times 0.1 \times 4$	match

UNIT	QUANTITY	BOOK	MEKHQ FORMULA	
CSR-V12 Corsair (5 t fuel)	Fuel	20 t hydrogen	5 × 4	match
Union DropShip (military)	Fuel	1.84/day → 27.6 t	military → 1.84 × 15	match
Mule DropShip (civilian, 11,200 t)	Fuel	4.22/day → 63.3 t	civilian <20000 → 4.22 × 15	match
Invader JumpShip (152,000 t)	Fuel	19.75/day → 59.25 t	<200000 → 19.75 × 3	match

Thirteen for thirteen. The DropShip and JumpShip burn-day tables in `getCraftMonthlyHydrogenUsage` are not approximations — they are the CamOps values, including the military/civilian split that the Union-versus-Mule comparison on p. 28 makes explicit.

The one that costs you money

The Mule is flagged military in mm-data, and that halves its fuel bill

MekHQ picks the civilian burn table via `getDesignType() != Dropship.MILITARY`. CamOps p. 28 explicitly calls the Mule *"the civilian Mule DropShip at 4.22 tons per burn-day (63.3 tons of hydrogen per month)"*. But **196 of 209 DropShips in mm-data carry `designtype 1` (MILITARY)**, the Mule among them.

So MekHQ charges the Mule **27.6 t / 414,000 CB** where the book says **63.3 t / 949,500 CB**. A 2.3× undercharge on the most common civilian freighter in the setting, and the same error scales up: a 100,000-ton Behemoth is billed as though it were a 1,000-ton military shuttle.

This is a **data defect, not a code defect** — the same `designtype` problem that made the field useless for classifying cargo haulers. Fixing the data fixes the fuel accounting for free.

Fuel

"Peacetime fuel usage rates are assumed to be four times the fuel capacity of the 'Mech, vehicle, fighter and infantry Forces... DropShips and WarShips are assumed to use 15 burn-days of fuel per month, while JumpShips and space stations are assumed to use 3 burn-days per month."

Combat vehicles at 10% engine mass, ×4

FAITHFUL

Validated against both the Bulldog and the Scorpion. Petrochemical for ICE, hydrogen for fuel cell, at the correct 1,000 / 15,000 CB per ton.

Unit.java:7632 getVehicleFuelCost

Large craft burn-day tables

FAITHFUL

DropShip, small craft, JumpShip and WarShip rates all match, including 15 burn-days for DropShips/WarShips and 3 for JumpShips. Space stations are covered because `SpaceStation` extends `Jumpship`.

Unit.java:7527 getCraftMonthlyHydrogenUsage

Infantry fuel uses platoon mass, not compartment mass

DIVERGES

The rule says jump, mechanized and motorized infantry burn **2% of the mass of the infantry compartment able to carry them**. The code uses 2% of the platoon's own weight. Different quantity entirely — and notably the spare-parts path right next to it *does* model compartments, so the two disagree with each other.

Unit.java:7664 getInfantryFuelCost

Support vehicles only half-covered

DIVERGES

Only `SupportTank` (and `LargeSupportTank` by inheritance) reads its listed fuel tonnage. `SupportVTOL` extends `VTOL`, so despite having its own `getFuelTonnage()` it falls through to the combat-vehicle engine-mass formula. Separately, "fuel as listed in their TRO" is exactly what megamek's `FuelType` enum encodes, but the code hardcodes 15,000/1,000 by engine type — so `ALCOHOL` (1,500) and `NATURAL_GAS` (1,200) are silently mispriced.

Unit.java:7634; megamek FuelType.java

IndustrialMeks use the vehicle approximation

DIVERGES

The rule wants a listed fuel tonnage from the TRO. Not MekHQ's fault: `Mek` has no `getFuelTonnage()` API in megamek at all, so there is nothing to read. Strict fidelity needs an upstream change first.

blocked on megamek Mek.java

Hydrogen production from fusion engines

EXCEEDS RAW

Not in the rule text, but firmly grounded in it — Arnold "decides he's definitely going to have to exploit the ability of military Forces to convert local water supplies into hydrogen." MekHQ models this as 10 t/day per fusion engine.

The implementation undercuts it, though. The pooled monthly total is passed whole to every unit and subtracted from each independently rather than being consumed once. One fusion engine yields 280 t/month against a DropShip's 27.6 t/month, so a single fusion-engined unit drives the hydrogen cost of every large craft, ASF and fusion fighter in the force to zero.

Accountant.java:787 → Unit.java:7507

Combat-operation fuel usage

NOT IMPLEMENTED

"To determine combat operation fuel usage, see page 33, SO, with the controlling player tracking the number of maintenance cycles that the units are in operation." Only the peacetime rate exists.

Spare parts

"Unlike ammunition, fuel and salaries, peacetime consumption of spare parts is generally abstract (unless an incident during Force operation results in distinct damage)."

CAMOPS P. 24

Tonnage rates and per-ton pricing

FAITHFUL

0.1% of mass for 'Mechs, fighters, vehicles, DropShips, WarShips and battle armor; 0.01% for JumpShips and space stations; 10,000 / 8,000 / 15,000 CB per ton by class. Seven separate worked examples reproduce exactly.

Unit.java:7339 getSparePartsCost

Foot infantry compartment

FAITHFUL

Hardcoded 3-ton compartment at 0.2% gives 0.006 t / 60 CB — precisely Arnold's figure. Jump (4 t) and motorized (6 t) follow the same pattern.

Mechanized infantry takes the wrong branch

DIVERGES

Mechanized infantry is charged `weight × 0.001 × 10000` — 0.1% of the platoon — while the rule puts all four conventional infantry types on 0.2% of the transport compartment. Every other infantry mode in the method uses the compartment approach.

Unit.java:7355

Quirk multipliers do not stack

DIVERGES

The rule is explicit that these apply **sequentially**, and gives a worked example: a 'Mech that is Easy to Maintain *and* uses Non-Standard Parts pays $10,000 \times 0.8 \times 2.0 = 16,000$ CB per ton. The code is an `if / else if` chain, so exactly one multiplier ever applies — the book's own example cannot be reproduced.

Unit.java:7379-7387

Rugged and Obsolete quirks

NOT IMPLEMENTED

Rugged ($\times 0.8$) is absent entirely. Obsolete ($\times 1.1$, plus 0.1 for every 20 years past the obsolescence date) is an explicit `// TODO Obsolete quirk`.

Real parts, real damage, real repairs

EXCEEDS RAW

This is where MekHQ leaves the book far behind. CamOps abstracts spare parts to a tonnage and a price. MekHQ models individual parts with condition, an acquisition system with availability rolls and delivery times, tech assignment, repair durations, and real inventory (`AmmoStorage`, `Armor`, `SVArmor`). The rulebook has no equivalent of any of it.

Extended parts modifier

EXCEEDS RAW

The rule multiplies spare parts by the Equipment Availability & Cost Table (p. 14). MekHQ instead offers an optional era-aware model — Star League tech scarcity 2860–3039, Clan tech premiums after 3049, XL/XXL and light engine surcharges, ICE tank penalties. Different mechanism, richer result, not RAW.

Unit.java:7391 `isUseExtendedPartsModifier`

Ammunition

"Divide a unit's ammunition tonnage and cost of ammunition by 4... This cost is based on standard ammunition even if the unit normally uses alternate ammunition types... This represents inexpensive training ammunition being expended."

CAMOPS P. 24

Quarter-capacity rate

FAITHFUL

The `× 0.25` is applied across all ammo-bearing parts, matching the rule.

Unit.java:7445

Priced at loaded munitions, not standard

DIVERGES

The rule is emphatic that peacetime ammunition is billed as *standard* training ammunition even when the unit habitually carries Artemis-compatible or armor-piercing loads. MekHQ charges `AmmoBin.getPricePerTon()` for whatever is actually loaded, so an Artemis-equipped force pays premium rates to run target practice.

Unit.java:7440 → AmmoBin.java:210

Current shots, not capacity

DIVERGES

`getStickerPrice()` is `pricePerTon × currentShots / shotsPerTon`, so the charge scales with what is presently in the bin. The rule bills a quarter of *capacity*. A force that returns from a battle with empty bins is billed almost nothing for the next month's training.

Infantry power-pack recharge rule

NOT IMPLEMENTED

Infantry with non-plasma energy weapons stationed alongside fission or fusion vehicles divide ammunition cost and weight **by 10**, because their packs recharge for free. Arnold's platoon drops from 11,200 CB to 1,120 CB on this rule alone — a 90% saving that MekHQ never applies.

Salaries

Quality multipliers

FAITHFUL

Green $\times 0.6$, Regular $\times 1.0$, Veteran $\times 1.6$, Elite $\times 3.2$ — exact. Anti-Mek training and specialist infantry multipliers are both present and correctly ordered.

CampaignOptions.java:157-164

Skill range and rank granularity

EXCEEDS RAW

MekHQ extends the table with Ultra-Green, Heroic ($\times 6.4$) and Legendary ($\times 12.8$). It also replaces the book's two rank formulas with a per-rank `payMultiplier` defined in the rank system, which is far more granular than $((\text{Rank TP}-3)/3 + 1)$.

Secondary roles

EXCEEDS RAW

Half the secondary role's base salary, with its own experience multiplier. The code comments note plainly that "CamOps doesn't cover secondary roles."

DropShip / JumpShip / WarShip crew are paid the same

NOT IMPLEMENTED

The Salary Table sets three distinct rates — Vessel Crewman (DropShip) 1,000, (JumpShip) 750, (WarShip) 1,200. MekHQ carries a literal `// TODO: distinguish DropShip, JumpShip, and WarShip crew`. A WarShip crew is currently underpaid by 20% and a JumpShip crew overpaid by a third, relative to RAW.

Person.java:5045

Two things worth deciding

Battle damage is not netted against the monthly allowance

REVIEW

The rule says: *"Battle damage that results in specific losses... may be debited from this monthly average spare part tonnage and costs before incurring extra costs."* RAW intends the abstract monthly spare-parts charge to **absorb** repair costs up to its value. MekHQ charges the monthly abstraction and then charges again for the real parts consumed in repair, with no offset between them. Worth confirming against the options matrix before calling it a double-charge, but I could find no netting logic.

The book contradicts itself on JumpShip burn-days

RULEBOOK ISSUE

The rule text and Arnold's Invader both use **3** burn-days for station-keeping ($19.75 \times 3 = 59.25$ t). Jason's Merchant on p. 28 uses **15** burn-days at a 0.1 G rate ($1.975 \times 15 = 29.625$ t). Both appear in the same section. MekHQ follows the rule text and the Invader, which is the defensible reading — no change recommended, but worth knowing if a user reports it.

Source: *BattleTech: Campaign Operations*, 2024 revised 5th print, pp. 24–29 (Stage 4: Determine Operating Expenses).
Implementation reviewed at `Unit.getSparePartsCost`, `Unit.getAmmoCost`, `Unit.getFuelCost`,
`Unit.getCraftMonthlyHydrogenUsage`, `Accountant.getFuelTotal`, `Person.getSalary` and `Finances.processNewDay`.
Worked-example figures taken from the completed worksheets of Arnold (p. 26), Hannah (p. 27) and Jason (p. 28).